```
/**
 * server/services/token-guardian-hardened.ts  (hardened)
 *
 * Fixes applied vs original:
 *  - Never overwrites a valid refresh_token with null.
 *    If Google only returns a new access_token, the existing refresh_token
 *    is preserved in the UPDATE.
 *  - Token values are redacted from all log output.
 *  - auditTokensOnBoot wired through DB readiness guard.
 *  - Recovery failure sets needs_reconnect, pauses publisher, stops retrying.
 */

import { db }          from '../db.js';
import { sql }         from 'drizzle-orm';
import { createLogger } from '../lib/logger.js';

const log = createLogger('token-guardian-hardened');

const redact = (token: string | null | undefined): string =>
  token ? `${token.slice(0, 4)}…[redacted]` : 'null';

// ─── Atomic token write ──────────────────────────────────────────────────────

export interface TokenSet {
  accessToken:     string;
  refreshToken?:   string | null;   // Optional — if absent, preserve existing
  tokenExpiresAt:  Date;
}

/**
 * Write OAuth tokens atomically.
 *
 * CRITICAL RULE: If `refreshToken` is absent or null in the payload,
 * the existing refresh_token in the DB is preserved. Google sometimes
 * returns only a new access_token; we must not null-out the refresh_token.
 */
export async function writeTokensAtomic(
  channelId: number,
  tokens:    TokenSet,
): Promise<{ success: boolean; error?: string }> {
  if (!tokens.accessToken) {
    return { success: false, error: 'accessToken is empty — write aborted' };
  }

  return db.transaction(async (tx) => {
    // Step 1: Backup existing tokens before overwriting.
    // Fix #10: only backup when BOTH access_token AND refresh_token are non-null.
    await tx.execute(sql`
```

```
  UPDATE channels
  SET
    access_token_backup  = access_token,
    refresh_token_backup = refresh_token,
    token_expires_backup = token_expires_at,
    token_backed_up_at   = NOW()
  WHERE id = ${channelId}
    AND access_token  IS NOT NULL
    AND refresh_token IS NOT NULL
`);

// Step 2: Write new tokens.
// If refreshToken is null/undefined, use COALESCE to keep the existing value.
await tx.execute(sql`
  UPDATE channels
  SET
    access_token       = ${tokens.accessToken},
    refresh_token      = COALESCE(${tokens.refreshToken ?? null}, refresh_token),
    token_expires_at   = ${tokens.tokenExpiresAt},
    last_token_refresh = NOW(),
    needs_reconnect    = false,
    reconnect_reason   = NULL
  WHERE id = ${channelId}
`);

// Step 3: Read back and verify — neither token is null
const verify = await tx.execute<{
  access_token: string | null;
  refresh_token: string | null;
}>(sql`
  SELECT access_token, refresh_token
  FROM channels
  WHERE id = ${channelId}
`);

const row = verify.rows[0];
if (!row?.access_token || !row?.refresh_token) {
  throw new Error(
    `Token write verification failed for channel ${channelId} — ` +
    `access_token=${redact(row?.access_token)}, ` +
    `refresh_token=${redact(row?.refresh_token)}`
  );
}

log.info(
  `[TokenGuardian] ✅ Tokens written atomically for channel ${channelId} — ` +
  `access=${redact(tokens.accessToken)}, ` +
  `refresh=${tokens.refreshToken ? 'updated' : 'preserved'}`
);
return { success: true };
```

```
  }).catch((err: Error) => {
    log.error(`[TokenGuardian] Atomic write failed for channel ${channelId}: ${err.message}`);
    return { success: false, error: err.message };
  });
}

// ─── Backup recovery ──────────────────────────────────────────────────────────

async function recoverFromBackup(
  channelId: number,
): Promise<{ recovered: boolean; reason: string; needsImmediateRefresh?: boolean }> {
  const result = await db.execute<{
    access_token_backup:  string | null;
    refresh_token_backup: string | null;
    token_backed_up_at:   Date | null;
    token_expires_backup: Date | null;  // Fix #11: also check expiry
  }>(sql`
    SELECT access_token_backup, refresh_token_backup,
           token_backed_up_at, token_expires_backup
    FROM channels
    WHERE id = ${channelId}
      AND access_token_backup  IS NOT NULL
      AND refresh_token_backup IS NOT NULL
  `);

  const backup = result.rows[0];
  if (!backup) {
    return { recovered: false, reason: 'No backup tokens exist — user must reconnect' };
  }

  // Fix #2: Do NOT reject recovery just because the backup ACCESS token is expired.
  // If the backup REFRESH token exists, restore it and queue an immediate refresh.
  // Only force needs_reconnect if the refresh token backup itself is missing.
  const refreshBackupPresent = !!backup.refresh_token_backup;
  if (!refreshBackupPresent) {
    return { recovered: false, reason: 'Backup refresh token missing — user must reconnect' };
  }

  // Check backup age (access token expiry is irrelevant — refresh token can renew it)
  const backedUpAt = backup.token_backed_up_at ? new Date(backup.token_backed_up_at) : null;
  if (!backedUpAt) {
    return { recovered: false, reason: 'Backup timestamp missing — cannot safely restore' };
  }
  const ageMs    = Date.now() - backedUpAt.getTime();
  const ageHours = Math.round(ageMs / 3_600_000);

  if (ageHours > 168) {
    return { recovered: false, reason: `Backup tokens are ${ageHours}h old — too stale to
restore` };
```

```
  }

  // Check if backup access token is already expired
  const accessExpired = backup.token_expires_backup
    ? new Date(backup.token_expires_backup) <= new Date()
    : true;  // treat null expiry as expired

  await db.execute(sql`
    UPDATE channels
    SET
      access_token       = access_token_backup,
      refresh_token      = refresh_token_backup,
      token_expires_at   = token_expires_backup,
      last_token_refresh = NOW(),
      needs_reconnect    = false,
      reconnect_reason   = NULL,
      token_recovery_note = ${'Recovered from backup at ' + new Date().toISOString() +
                             ' (backup was ' + ageHours + 'h old' +
                             (accessExpired ? ', access token expired — refresh required)' :
')')}
    WHERE id = ${channelId}
  `);

  const reason = accessExpired
    ? `Restored refresh token from backup (${ageHours}h old). Access token expired — immediate
refresh required.`
    : `Restored from ${ageHours}h-old backup`;

  log.warn(
    `[TokenGuardian] Restored backup tokens for channel ${channelId} ` +
    `(${ageHours}h old, accessExpired=${accessExpired})`
  );
  return { recovered: true, reason, needsImmediateRefresh: accessExpired };
}

// ─── Mark channel as needing reconnect ──────────────────────────────────────

async function markNeedsReconnect(channelId: number, reason: string): Promise<void> {
  await db.execute(sql`
    UPDATE channels
    SET needs_reconnect = true, reconnect_reason = ${reason}
    WHERE id = ${channelId}
  `);
  log.error(
    `[TokenGuardian] Channel ${channelId} marked needs_reconnect. ` +
    `Reason: ${reason}. YouTube publishing PAUSED until user reconnects.`
  );
}

// ─── Boot audit ─────────────────────────────────────────────────────────────
```

```
export interface TokenHealthReport {
  channelId:   number;
  channelName: string;
  status:      'healthy' | 'null_recovered' | 'null_no_backup' | 'expiring_soon';
  action:      string;
}

/**
 * Run at Stage 4 of startup (YouTube Connection Health).
 * Detect null tokens, attempt backup recovery, mark needs_reconnect if unrecoverable.
 * Does NOT retry repair endlessly — one attempt then stops.
 */
export async function auditTokensOnBoot(): Promise<TokenHealthReport[]> {
  const channels = await db.execute<{
    id:              number;
    channel_name:    string;
    access_token:    string | null;
    refresh_token:   string | null;
    token_expires_at: Date | null;
    needs_reconnect: boolean;
  }>(sql`
    SELECT id, channel_name, access_token, refresh_token,
           token_expires_at, needs_reconnect
    FROM channels
    WHERE platform = 'youtube'
      AND LOWER(channel_name) NOT LIKE '%demo%'
      AND LOWER(channel_name) NOT LIKE '%test%'
      AND LOWER(channel_name) NOT LIKE '%reviewer%'
  `);

  const reports: TokenHealthReport[] = [];

  for (const ch of channels.rows) {
    const isNull      = !ch.access_token || !ch.refresh_token;
    const expiresAt   = ch.token_expires_at ? new Date(ch.token_expires_at) : null;
    const expiresInH  = expiresAt
      ? Math.round((expiresAt.getTime() - Date.now()) / 3_600_000)
      : -1;

    if (isNull) {
      const recovery = await recoverFromBackup(ch.id);
      if (recovery.recovered) {
        reports.push({
          channelId:   ch.id,
          channelName: ch.channel_name,
          status:      'null_recovered',
          action:      `Tokens restored. ${recovery.reason}.` +
                       (recovery.needsImmediateRefresh
                         ? ' ⚠ Immediate token refresh required.'
```

```
                        : ' Refresh queued normally.'),
        });
      } else {
        await markNeedsReconnect(ch.id, recovery.reason);
        reports.push({
          channelId:   ch.id,
          channelName: ch.channel_name,
          status:      'null_no_backup',
          action:      `RECONNECT REQUIRED: ${recovery.reason}`,
        });
      }
    } else if (expiresInH >= 0 && expiresInH < 2) {
      reports.push({
        channelId:   ch.id,
        channelName: ch.channel_name,
        status:      'expiring_soon',
        action:      `Token expires in ${expiresInH}h — refresh queued`,
      });
    } else {
      reports.push({
        channelId:   ch.id,
        channelName: ch.channel_name,
        status:      'healthy',
        action:      `Tokens valid${expiresInH >= 0 ? `, expires in ${expiresInH}h` : ''}`,
      });
    }
  }

  return reports;
}

/**
 * Guard for publisher — call before any YouTube API call.
 * Fix #12: verifies actual token presence, not only needs_reconnect flag.
 * Fix #3:  also checks token_expires_at — if access token is expired but
 *          refresh token is present, returns { canPublish: false, shouldRefresh: true }
 *          so the caller can refresh first rather than giving up.
 */
export async function channelCanPublish(channelId: number): Promise<{
  canPublish:     boolean;
  shouldRefresh:  boolean;
  reason:         string;
}> {
  const result = await db.execute<{
    needs_reconnect:  boolean;
    reconnect_reason: string | null;
    access_token:     string | null;
    refresh_token:    string | null;
    token_expires_at: Date | null;
  }>(sql`
```

```
    SELECT needs_reconnect, reconnect_reason,
           access_token, refresh_token, token_expires_at
    FROM channels WHERE id = ${channelId}
  `);

  const ch = result.rows[0];
  if (!ch) {
    log.warn(`[TokenGuardian] Channel ${channelId} not found — blocking publish`);
    return { canPublish: false, shouldRefresh: false, reason: 'Channel not found' };
  }

  if (ch.needs_reconnect) {
    log.warn(`[TokenGuardian] Channel ${channelId} needs reconnect — blocking
(${ch.reconnect_reason})`);
    return { canPublish: false, shouldRefresh: false, reason: ch.reconnect_reason ??
'needs_reconnect' };
  }

  if (!ch.refresh_token) {
    log.warn(`[TokenGuardian] Channel ${channelId} missing refresh_token — blocking`);
    return { canPublish: false, shouldRefresh: false, reason: 'refresh_token is null' };
  }

  if (!ch.access_token) {
    // No access token but refresh token present — can refresh
    log.warn(`[TokenGuardian] Channel ${channelId} missing access_token — refresh required`);
    return { canPublish: false, shouldRefresh: true, reason: 'access_token is null — refresh
required' };
  }

  // Check expiry
  if (ch.token_expires_at) {
    const expiresAt  = new Date(ch.token_expires_at);
    const expiresInMs = expiresAt.getTime() - Date.now();
    if (expiresInMs <= 0) {
      // Expired — can refresh using refresh_token
      log.warn(`[TokenGuardian] Channel ${channelId} access token expired — refresh required`);
      return { canPublish: false, shouldRefresh: true, reason: 'access_token expired — refresh
required' };
    }
    if (expiresInMs < 5 * 60 * 1000) {
      // Expires in < 5 min — refresh proactively before the call
      return { canPublish: false, shouldRefresh: true, reason: 'access_token expiring in < 5 min
— refresh first' };
    }
  }

  return { canPublish: true, shouldRefresh: false, reason: 'ok' };
}
```

```
// ─── Publisher usage pattern ──────────────────────────────────────────────────
//
//   const guard = await channelCanPublish(channelId);
//   if (!guard.canPublish) {
//     if (guard.shouldRefresh) {
//       const refreshed = await refreshChannelToken(channelId);  // your existing refresh fn
//       if (!refreshed) {
//         await markNeedsReconnect(channelId, 'Token refresh failed at publish time');
//         return; // block
//       }
//       // proceed with YouTube API call after successful refresh
//     } else {
//       return; // hard block — needs human reconnect
//     }
//   }
```